

Learning Lean 4

Joshua W. Kelly

2026-7-19

Abstract

Working notes and exercises from learning the Lean 4 theorem prover, following *An Introduction to Lean 4* by Cosme Llópez and Gong. Covers basic syntax and natural-deduction proofs, paired with a runnable Lean library.

Contents

1	Foundations	1
1.1	Knowledge graph	1
1.2	How to read the two tracks	1
2	Basic Syntax	2
2.1	Basic syntax	2
2.1.1	Comments	2
2.1.2	Inspecting terms and types with <code>#check</code>	2
2.1.3	Printing definitions with <code>#print</code>	2
2.1.4	Definitions and functions	2
3	Natural Deduction	3
3.1	Natural deduction	3
3.1.1	5.5 Reduction to the absurd. $P \Rightarrow Q, \neg Q \vdash \neg P$	3
3.1.2	Conjunction introduction. $P, P \Rightarrow Q \vdash P \wedge Q$	3
4	Linear Algebra	4
4.1	Linear algebra	4
4.1.1	Vectors	4
4.1.2	Matrices	4
4.1.3	Linear maps	5
4.1.4	A worked computation	5
4.1.5	Exercises	5
5	Accessibility	6
5.1	Accessibility & universal design	6
5.1.1	Principles	6
5.1.2	Available formats	6
5.1.3	Feedback	7
	References	8

1 Foundations

This section maps the skills and background knowledge that build on one another while learning Lean 4¹ and the mathematics it is used to formalize. It is a roadmap rather than a chapter of exercises: the two tracks – *mathematics* and *Lean 4* – develop in parallel and meet where proofs get formalized against Mathlib².

1.1 Knowledge graph

The graph below shows prerequisite relationships (an arrow $A \dashrightarrow B$ reads “A supports B”). Read it bottom-up from the two entry points – propositional logic on the mathematics side and basic syntax on the Lean side – toward formalized mathematics, where the tracks converge.

1.2 How to read the two tracks

Mathematics. Propositional and predicate logic together with basic proof techniques (direct proof, contradiction, induction) are the bedrock. Set-theoretic language – functions, relations, and cardinality – makes those proofs precise. From there, abstract algebra provides the structures (groups, rings, fields) that linear algebra and analysis specialize.

Lean 4. The language side starts with syntax and the term/type distinction, then the *propositions-as-types* correspondence, which is where logic on the mathematics side connects: a proof of P is a term of type P . Tactics automate the construction of those terms, and natural deduction is the first substantial place both tracks are exercised together.

Convergence. Both tracks meet at Mathlib, Lean’s community library, where formalizing real mathematics requires fluency in the mathematical structure *and* in the Lean idioms that encode it. That convergence – formalized mathematics – is the goal the rest of this book works toward.

2 Basic Syntax

2.1 Basic syntax

The runnable version of this chapter lives in `LearningLean/BasicSyntax.lean`.

2.1.1 Comments

```
-- This is a single-line comment.

/-
This is a multi-line comment.
Here is another line.
-/-
```

2.1.2 Inspecting terms and types with `#check`

`#check` asks Lean for the type of an expression without evaluating it.

```
#check true           -- Bool
#check 42             -- Nat
#check "Hello, World!" -- String
#check Type           -- Type 1
#check Nat            -- Type
#check ['h', 'e', 'l', 'l', 'o'] -- List Char
```

2.1.3 Printing definitions with `#print`

`#print` shows how something is defined.

```
#print Bool
```

Type itself cannot be printed with `#print`, because it is a type rather than a definition.

2.1.4 Definitions and functions

```
def pi : Float := 3.1415926
```

```
#check pi
```

```
-- An anonymous function (lambda) taking two natural numbers.
```

```
#check λ (a b : Nat) => a + b
```

3 Natural Deduction

3.1 Natural deduction

The Lean proofs for this chapter live in `LearningLean/NaturalDeduction.lean`, inside the `Exercises` namespace.

3.1.1 5.5 Reduction to the absurd. $P \Rightarrow Q, \neg Q \vdash \neg P$

This is a very useful technique. Validity of $P \Rightarrow Q, \neg Q \vdash \neg P$ is proved with:

1	$P \Rightarrow Q$	
2	$\neg Q$	
3	P	H
4	Q	$\Rightarrow$ 1,3
5	$\neg Q$	IT 2
6	$\neg P$	$\neg$ I 3,4,5

The sub-derivation (lines 3–5) discharges the hypothesis P : assuming P , [$\Rightarrow$ -elimination] on lines 1 and 3 gives Q , while line 2 reiterates $\neg Q$. The contradiction lets us introduce $\neg P$ ($\neg$ -introduction over lines 3, 4, 5).

In Lean this is theorem `T55`, where $\neg P$ is definitionally $P \rightarrow \text{False}$,¹ so the whole proof is just function composition:

```
theorem T55 (h1 : P → Q) (h2 : ¬Q) : ¬P :=  
  (h2 ∘ h1)
```

3.1.2 Conjunction introduction. $P, P \Rightarrow Q \vdash P \wedge Q$

Theorem `T51` builds the pair (proof of P , proof of Q) directly, using the anonymous-constructor notation `{_, _}`:

```
theorem T51 (h1 : P) (h2 : P → Q) : P ∧ Q :=  
  {h1, h2 h1}
```

¹This is the standard definition in Lean's core library: `Not p` unfolds to `p → False`, so a proof of $\neg P$ is exactly a function turning a proof of P into a proof of `False`.

4 Linear Algebra

4.1 Linear algebra

A scratch chapter for linear-algebra notes – enough structure to flesh out later, and enough variety (numbered equations, a table, admonitions, and interleaved Lean) to show the book template in action. The runnable code lives in `LearningLean/LinearAlgebra.lean`.

Note

This chapter is a work in progress. Sections marked **To flesh out** are placeholders.

4.1.1 Vectors

A vector in R^n is an ordered list of n real numbers. We write column vectors as $\vec{u} = (u_1, \dots, u_n)$ with each $u_i \in R$.

The **dot product** of $\vec{u}, \vec{v} \in R^n$ is the scalar

$$\vec{u} \cdot \vec{v} = \sum_{i=1}^n u_i v_i.$$

For a concrete computation, take $\vec{u} = (1, 2, 3)$ and $\vec{v} = (4, 5, 6)$. Then by equation (1),

$$\vec{u} \cdot \vec{v} = 1 \cdot 4 + 2 \cdot 5 + 3 \cdot 6 = 32.$$

In Lean, a vector is a function `Fin n → ℝ`, and `![...]` is notation for one. The dot product and the matrix operations below follow the spellings used in `Mathlib`²:

```
def u : Fin 3 → ℝ := ![1, 2, 3]
def v : Fin 3 → ℝ := ![4, 5, 6]
```

```
#check u [?] v v      -- `[?] v` is Mathlib's dot product, of type ℝ
```

4.1.2 Matrices

An $m \times n$ matrix $A \in R^{m \times n}$ is a rectangular array of scalars. For example,

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \in R^{2 \times 2}.$$

²Mathlib writes the dot product as `[?] v` (`\cdot v`) rather than reusing `·`, to keep it distinct from scalar multiplication.

The **matrix–vector product** $A\vec{x}$ takes a vector $\vec{x} \in R^n$ to a vector in R^m . With $\vec{x} = (5, 6)$:

$$A\vec{x} = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \begin{bmatrix} 5 \\ 6 \end{bmatrix} = \begin{bmatrix} 1 \cdot 5 + 2 \cdot 6 \\ 3 \cdot 5 + 4 \cdot 6 \end{bmatrix} = \begin{bmatrix} 17 \\ 39 \end{bmatrix}.$$

A few operations and their Mathlib spellings:

Operation	Math	Lean / Mathlib
Dot product	$\vec{u} \cdot \vec{v}$	<code>u</code> <code>·</code> <code>v</code>
Matrix–vector product	$A\vec{x}$	<code>A</code> <code>*_v</code> <code>x</code>
Matrix product	AB	<code>A</code> <code>*</code> <code>B</code>
Transpose	A^T	<code>A</code> <code>ᵀ</code>

```
def A : Matrix (Fin 2) (Fin 2) ℝ := !![1, 2; 3, 4]

#check A *_v ![5, 6] -- matrix-vector product, of type `Fin 2 → ℝ`
```

4.1.3 Linear maps

A **linear map** $T : R^n \rightarrow R^m$ is a function satisfying $T(\alpha\vec{u} + \vec{v}) = \alpha T(\vec{u}) + T(\vec{v})$ for all scalars α and vectors $\vec{u}, \vec{v}$. Every such map is represented by a matrix, and matrix multiplication corresponds to composition of the maps.

Note

To flesh out: state and prove that `Matrix.mulVec A` is a `LinearMap`, and connect it to `Matrix.toLin`.

4.1.4 A worked computation

To flesh out. A step-by-step example – e.g. solving $A\vec{x} = \vec{b}$ by row reduction – narrated alongside the corresponding Lean definitions, so the computation and its formalization sit side by side.

4.1.5 Exercises

Note

To flesh out. Add exercises here, e.g.:

1. Show the dot product is symmetric: $\vec{u} \cdot \vec{v} = \vec{v} \cdot \vec{u}$.
2. Prove $(A^T)^T = A$ in Lean using `Matrix.transpose_transpose`.

5 Accessibility

5.1 Accessibility & universal design

This book is written with **universal design** in mind: the goal is a single body of content that works for the widest possible range of readers, without anyone needing a separate “accessible version.” Accessibility here is not an afterthought bolted onto the finished text – it shapes how the source is authored and how it is published.

5.1.1 Principles

- **Equitable use.** The same content is offered in many formats so readers can choose the one that fits their tools, bandwidth, and reading style – screen reader, e-reader, print, or plain text.
- **Perceivability.** Figures carry text alternatives, mathematics is written as real markup (not images), and color is never the only way meaning is conveyed. The Fitch derivation is rendered as SVG with descriptive context in the surrounding prose.
- **Structure over styling.** Headings, lists, and tables use semantic markup so assistive technology can navigate the document. Every chapter has a clear heading hierarchy.
- **Readable mathematics.** Formulas are authored in LaTeX-style markup and rendered as MathML/accessible math in HTML, so screen readers and braille displays can interpret them rather than treating equations as opaque pictures.
- **Runnable, not locked away.** Lean source is linked to the live editor so proofs can be explored interactively, and the same code is available as plain text for offline reading.
- **Tolerance and flexibility.** Low-bandwidth and offline readers are served by the plain `.txt` and Markdown versions; print readers by the PDF; e-reader users by EPUB.

5.1.2 Available formats

The same content is published in several formats. Choose whichever suits your reading tools:

Format	Best for	Status
HTML	Reading online with accessible, navigable math	Available

Format	Best for	Status
PDF	Print and fixed-layout reading (built via LaTeX/Tectonic)	Available
Typst	Alternate PDF engine with a custom title page + TOC	Available
DOCX	Editing, annotation, and Office/assistive-tool workflows	Available
Markdown (.md)	Plain, portable source that reads well in any editor	Available
Plain text (.txt)	Maximum-compatibility, low-bandwidth, distraction-free reading	Planned
EPUB3	Reflowable reading on e-readers, with adjustable font size	Planned
AI Audio	Listening / audiobook-style narration	Planned

Note

Planned formats. Plain-text, EPUB3, and an AI-narrated audio edition are on the roadmap so the book can be read at any bandwidth, on e-readers, and listened to as well as read – supporting readers who prefer or rely on those modes.

5.1.3 Feedback

Accessibility is never “done.” If any part of this book is difficult to read with your tools – a missing figure description, math that doesn’t render for your screen reader, or a format you need that isn’t listed – please open an issue so it can be fixed.

References

1. Moura, L. de & Ullrich, S. The Lean 4 Theorem Prover and Programming Language. in *Automated Deduction – CADE 28* vol. 12699 625–635 (2021).
2. The mathlib Community. The Lean Mathematical Library. in *Proceedings of the 9th ACM SIGPLAN International Conference on Certified Programs and Proofs (CPP 2020)* 367–381 (2020). doi:10.1145/3372885.3373824.